TP3 - Exercice 2 : Questions théoriques

Développement mobile (Flutter) - 2026-2027 - Oumaima Iben Elwefi

Question 2 : Différence entre StatelessWidget et StatefulWidget

	StatelessWidget	StatefulWidget
État	Aucun état qui change. L'affichage reste identique.	Possède un état (state) qui peut changer pendant l'exécution.
Mise à jour	Se reconstruit seulement si le parent lui donne de nouvelles valeurs.	Se reconstruit quand on appelle setState().
Exemple	Une page statique avec titre, image et bouton (HomePage).	Un compteur qui augmente à chaque clic, une case à cocher, un formulaire.

Question 3 : Rôle de Scaffold, et pourquoi on ne peut pas le comparer à StatelessWidget

Scaffold est un widget prêt à l'emploi qui fournit la structure de base d'une page Material : `appBar`, `body`, `floatingActionButton`, `drawer`, `bottomNavigationBar`, etc. Il décrit **ce qu'il y a dans la page**.

StatelessWidget est une **classe de base** que l'on étend pour créer ses propres widgets (par exemple `class HomePage extends StatelessWidget`). Elle décrit **comment on construit un widget**.

On ne peut pas les comparer directement car ils ne sont pas de même nature : l'un est un élément d'interface (structure de page), l'autre est un modèle pour fabriquer des widgets. En pratique, on les combine : le StatelessWidget retourne un Scaffold dans sa méthode `build`.

Question 4 : Navigator.push, Navigator.pop et la pile de navigation

- `Navigator.push(context, route)` ouvre une nouvelle page **au-dessus** de la page actuelle.
- `Navigator.pop(context)` ferme la page actuelle et revient à la page précédente.

La **pile (stack)** de navigation est une pile de pages, comme une pile d'assiettes : la dernière page ajoutée est au sommet et c'est elle qui est affichée. *push* ajoute une page au sommet, *pop* retire la page du sommet et affiche celle qui est en dessous.

```
Départ          : [HomePage]
Après push      : [HomePage, SecondRoute]
Après pop       : [HomePage]
```

Question 5 : Pourquoi éviter d'imbriquer deux MaterialApp ?

MaterialApp est la racine de l'application. Elle configure une seule fois, pour toute l'application, le thème, le *Navigator*, les routes et la localisation. En imbriquer une deuxième crée un deuxième Navigator et un deuxième thème, ce qui provoque des problèmes :

- `push` et `pop` peuvent agir sur le mauvais Navigator, et le retour arrière se comporte de façon étrange.
- Le thème et les réglages du parent sont perdus dans la MaterialApp imbriquée.
- L'application est plus lourde et plus difficile à déboguer.

Solution : une seule MaterialApp tout en haut, et chaque autre page retourne simplement un Scaffold.